Metis-Auto v2

A measured LLM router built on UniRoute

Proposal v2 • Rahnema College 2026

19 October 2026

Abstract

Chatbot platforms pay very different prices for different AI models, while many messages are simple. The course asks for a router that picks a cheap model when that is enough and a strong model when it is not, without sacrificing quality. We build the router exactly as in UniRoute (Jitkrittum et al., ICLR 2026), following our specification `UNIROUTE_PIPELINE.md`: a large *unlabeled* set of real prompts is grouped into K clusters; each candidate model answers a small labeled validation set, graded by an LLM judge checked against humans; each model's fail rate per cluster becomes its fingerprint; and each new message goes to the model with the lowest fail rate plus $\lambda \times$cost in its cluster. New models are added without retraining. The router sits inside a gateway with hard rules, a semantic cache, failover, escalation and traces. We wait for the full dataset before any paid API call. Estimated API cost: **$17–50** (**$60** planned), recomputed from the full data first.

1 Why version 2

Version 1 (17 October) used the 377 sample turns for everything: clustering, report cards, choosing settings and the final numbers. Comparing it with the paper and our pipeline specification showed three problems.

Problem in v1	**Fix in v2**
Clusters and report cards came from the *same* labeled data, which overfits	Clusters from a large *unlabeled* train split; report cards only from the validation split
K and λ were tuned on the same data we reported on	A **locked test set**, used once at the end
Answers were graded against production's answer, which came from 18 different models	Pointwise grading, or pairwise against a fixed *anchor* model, chosen by a **pilot**

The paper's setting needs thousands of unlabeled prompts and about 700 labeled ones; only the full dataset can provide that.

2 Problem

Need (course brief, dataset)	Why it is hard
Users should not choose models ("AI decision fatigue")	The system must judge each message automatically and cheaply
Cut cost without hurting quality	Quality is easy to claim and hard to measure
Mostly Persian traffic	Judges and routers are checked mainly in English; we found no router research on Persian
Four very different usage types	The design must generalize, not be tuned to one chatbot
Reliability and observability	Provider outages and rate limits; every decision must be explainable

3 Solution overview

The **offline build** (specification steps S1–S12) splits the data, clusters the unlabeled train split, generates and grades answers on the validation and test splits, computes fingerprints, chooses K and λ, evaluates once on the test split and packages a versioned **router artifact**. The **online gateway** applies hard rules, embeds the message, checks the semantic cache, routes with UniRoute, calls the model with failover, runs code checks with escalation, and records a trace (Figure 1).

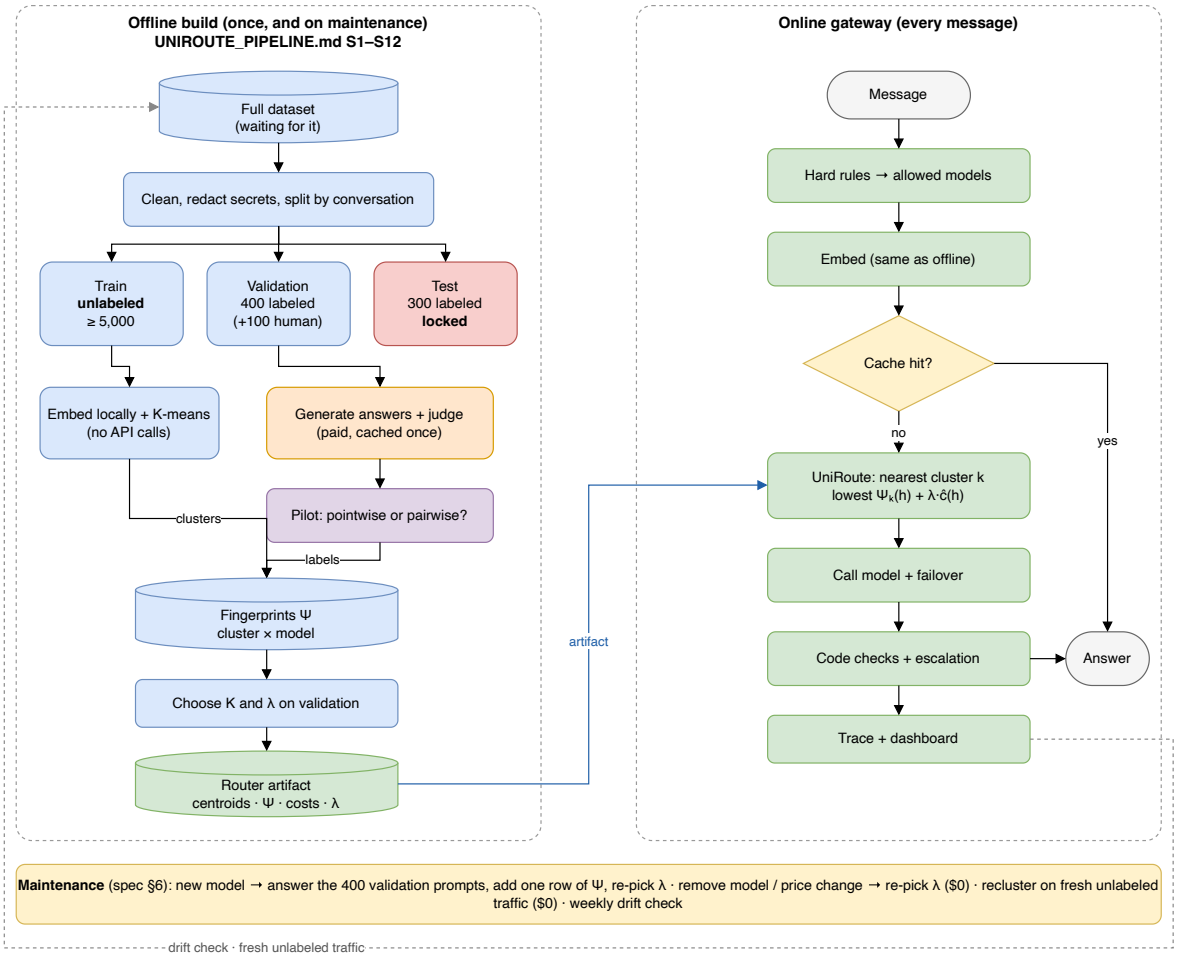


Figure 1: Offline build, online gateway, and maintenance.

4 How UniRoute works

Figure 2 shows every UniRoute step on one page. The step codes (S1–S12, R1–R4, M1–M4) match the specification; a full walkthrough is in `docs/uniroute-explained.md`.

Band	Steps	What happens
1 Prepare	S1, S2, S3	Clean prompts (last message + 2 turns); split by conversation into **train (unlabeled)**, validation 400, test 300 (locked); register models, anchor and costs $\hat{c}(h) \in [0, 1]$
2 Prompt types (free)	S4, S5	Embed the train prompts with a local multilingual model; K-means finds K clusters (centroids) without labels
3 Fingerprints (paid)	S6, S7, S8	Every model answers validation and test once; the judge turns each answer into a loss (pointwise or pairwise, chosen by the pilot); $\Psi_k(h)$ = model h's average loss on the validation prompts of cluster k
4 Settings	S9, S10	K by split-validation; λ for a quality target or a cost budget; validation only
5 Evaluate and ship	S11, S12	One evaluation on the locked test set against always-best, always-cheapest, ZeroRouter and K-NN; a versioned router artifact
6 Route (online)	R1–R4	Embed, find the nearest cluster k^*, score $s(h) = \Psi_{k^*}(h) + \lambda\,\hat{c}(h)$, route to the lowest (fallback: next lowest)
7 Maintain	M1–M4	Add a model (one new row of Ψ), remove a model, price change, traffic drift; **no retraining**

Worked example (specification, Appendix B; $\lambda = 0.4$).

Prompt type	S (strong, anchor)	M (medium)	C (cheap)	Route to
Casual chat	$0.50 + 0.40 = 0.90$	$0.52 + 0.12 = 0.64$	$0.55 + 0.02 = \mathbf{0.57}$	**C**
Persian literature	$0.50 + 0.40 = \mathbf{0.90}$	$0.80 + 0.12 = 0.92$	$0.92 + 0.02 = 0.94$	**S**

In chat the cheap model is almost as good as the anchor and 20 times cheaper, so it wins. In literature its loss is much higher, so paying for S is worth it. With $\lambda = 0$ the router always picks S; with a very large λ it always picks C.

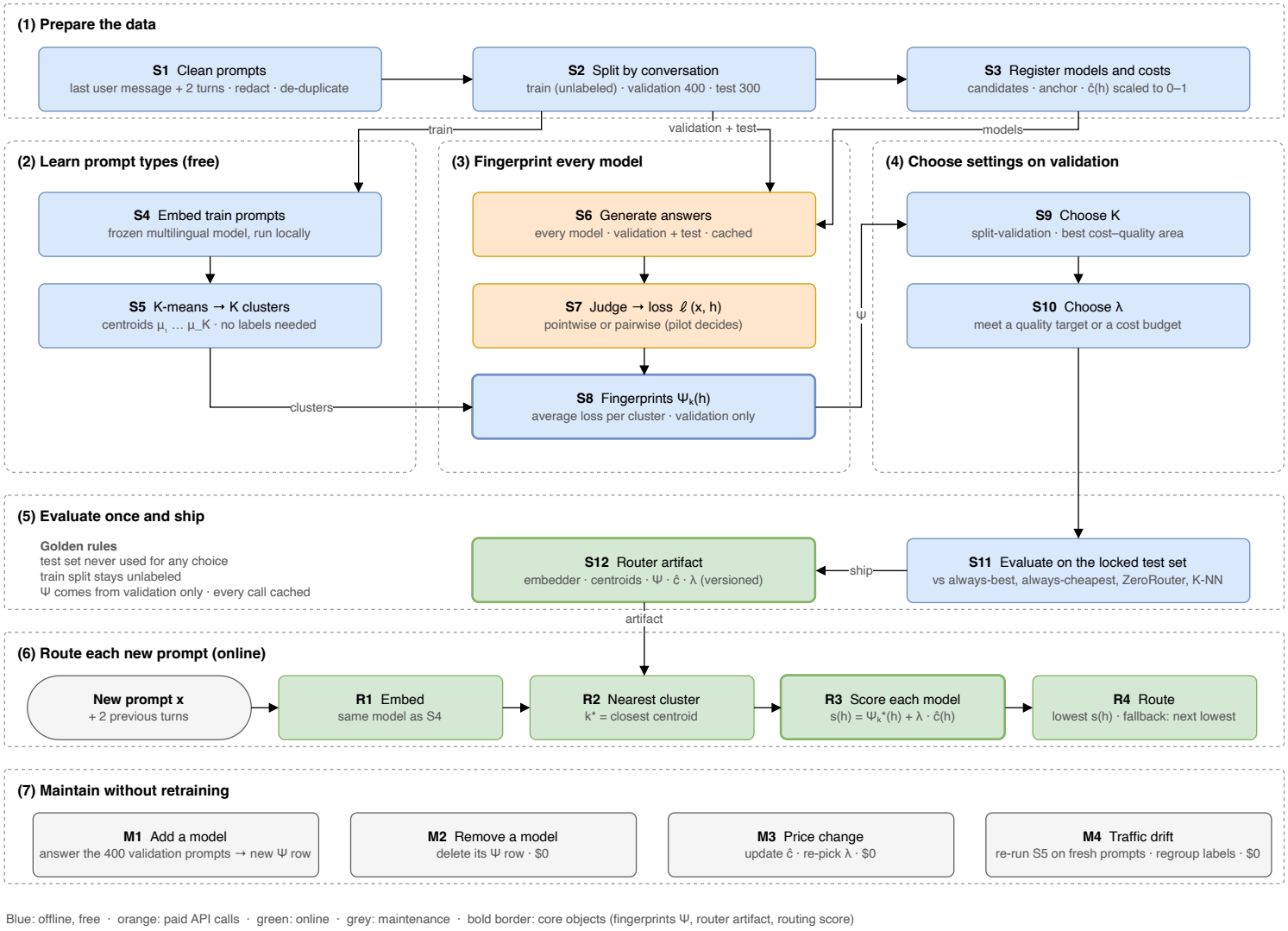


Figure 2: UniRoute, every step (S1–S12 offline, R1–R4 online, M1–M4 maintenance). Blue: offline and free; orange: paid API calls; green: online; grey: maintenance; bold border: core objects (fingerprints Ψ, router artifact, routing score).

5 Offline build

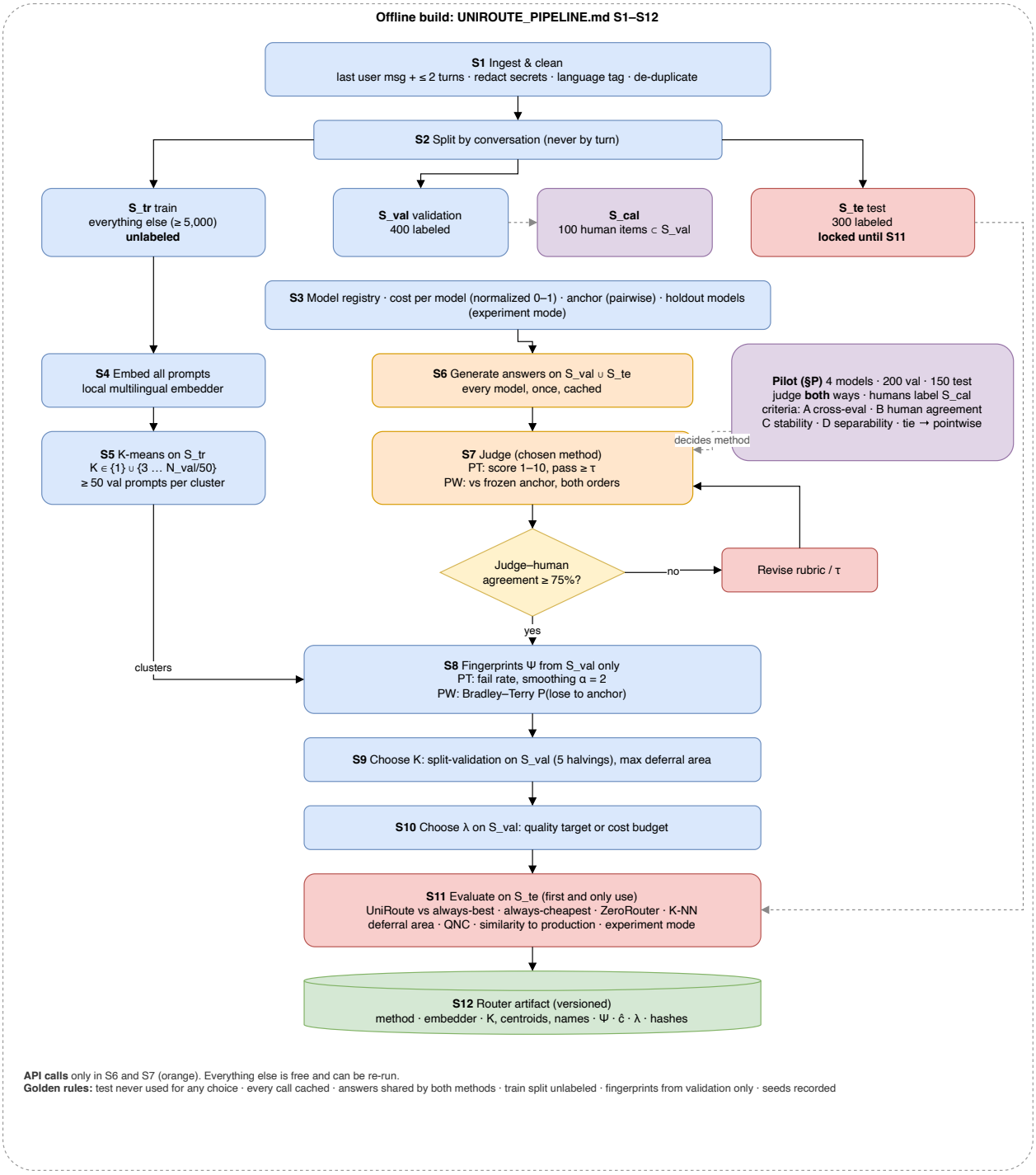


Figure 3: Offline build (S1–S12). Only S6 and S7 make paid API calls.

Step	What	API
S1 Clean	Routable prompt = last user message + up to 2 earlier turns; redact secrets; language tag; de-duplicate	0
S2 Split	By conversation: train (unlabeled, ideally $\geq$ 5,000), validation 400, test 300 (locked), calibration 100 $\subset$ validation	0
S3 Models	5 candidates; cost per model from real token counts, normalized to $[0, 1]$; anchor = strongest	0
S4–S5 Cluster	Local multilingual embedder; K-means on train for $K \in \{3, \ldots, \lfloor N_{\text{val}}/50 \rfloor\}$ plus $K = 1$; ≥ 50 validation prompts per cluster	0
S6 Generate	Every model answers validation and test once, with its chatbot's own system prompt and context; cached	paid
S7 Judge	Pointwise (score 1–10, pass if $\geq \tau$) or pairwise against the anchor (both orders); judge–human agreement $\geq 75\%$	paid
S8 Fingerprints	Fail rate per cluster with smoothing $\alpha = 2$ ($\alpha = 0$ is the paper's eq. 12), or Bradley–Terry P(lose to anchor)	0
S9–S10 Settings	K by split-validation on the validation set; λ for a quality target or a cost budget	0
S11 Evaluate	On the locked test set only (Section 7)	0
S12 Package	Router artifact: embedder, centroids, cluster names, fingerprints, costs, λ	0

Candidate models (5): `gemini-2.5-flash-lite`, `deepseek-v4-flash`, `gpt-4o-mini`, `gemini-2.5-flash`, `claude-sonnet-4-6` (strong; anchor). **Judge:** a strong Persian-capable model outside the pool, temperature 0, blind to model names.

The pilot. Before the full build we run a pilot on part of the full data: 4 models, 200 validation and 150 test prompts, *both* grading methods on the same answers, and 100 human labels. It picks the method by (A) routing quality under both measures, (B) agreement with humans, (C) stability of the fingerprints and (D) whether the labels can tell models apart. Ties go to pointwise, which is cheaper. Everything the pilot pays for is reused.

Routing rule (paper eq. 8 and 11). For a message x in cluster $k(x)$:

$$r(x) \;=\; \arg\min_{h} \; \Psi_{k(x)}(h) + \lambda \cdot \hat{c}(h),$$

where $\Psi_k(h)$ is model h's fail rate (or loss against the anchor) in cluster k and $\hat{c}(h)$ is its normalized cost.

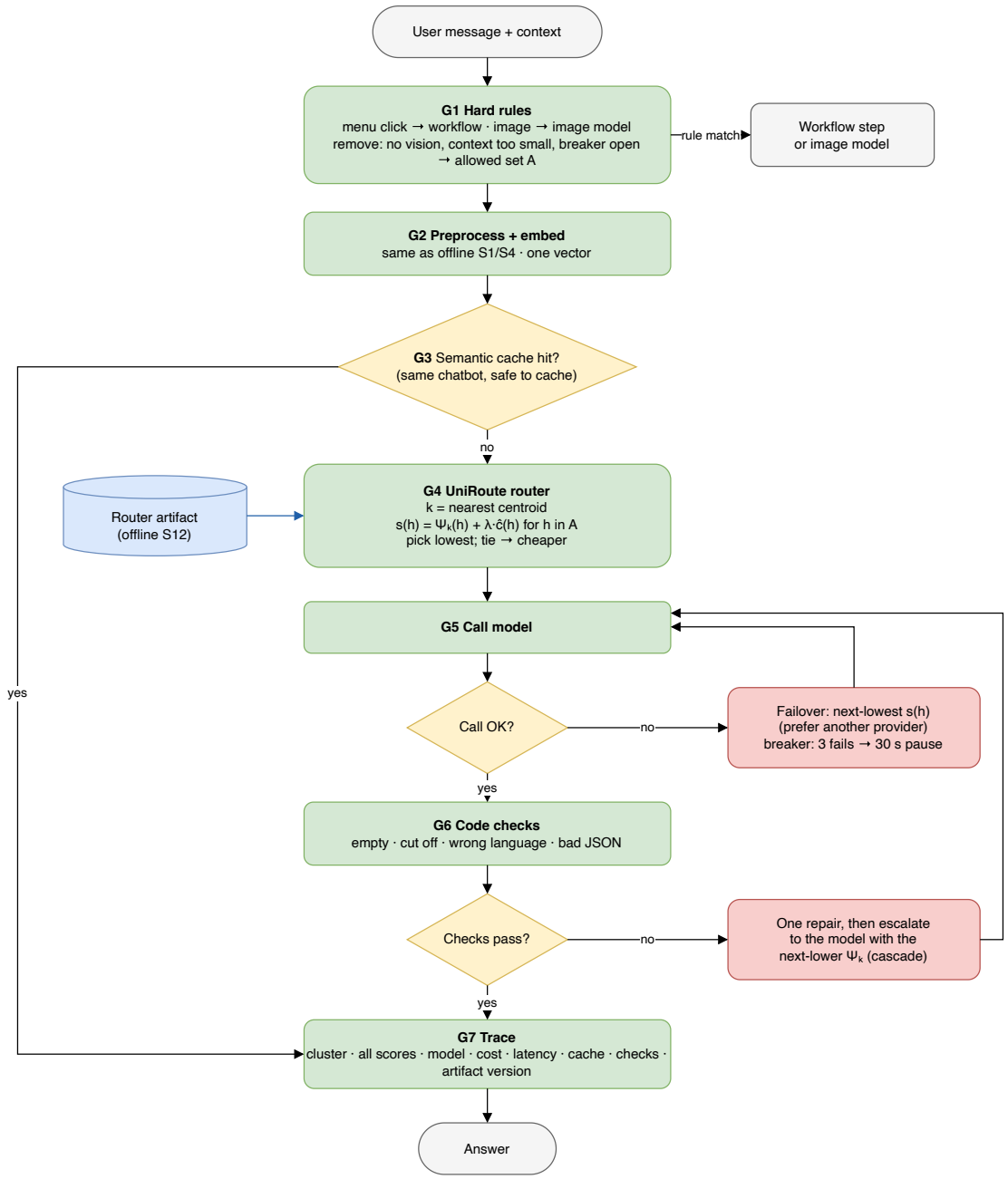


Figure 4: Online path for one message. Red boxes are recovery steps.

6 Online gateway

Stage	What it does
G1 Hard rules	Menu clicks go to the workflow step, images to the image model; remove models without vision, with too small a context window, or with an open circuit breaker
G2 Embed	Same preprocessing and embedder as offline; one vector
G3 Cache	Same chatbot and language only; never follow-ups, personal or sensitive content; threshold for $\leq 2\%$ false hits
G4 Router	Nearest cluster; lowest $\Psi_k(h) + \lambda\hat{c}(h)$ among the allowed models
G5 Call	On error, the next-lowest score, preferably another provider; circuit breaker (3 failures, then a 30 s pause)
G6 Checks	Free code checks (empty, cut off, wrong language, bad JSON), one repair, then escalate to the next-lower-loss model (the brief's cascade)
G7 Trace	Cluster, all scores, model, cost, latency, cache hit, checks, artifact version

7 Evaluation

All results come from the **locked test set**, used once.

Strategy	What it is
Always-best	The strongest model for everything (the brief's premium baseline)
Always-cheapest	The cheapest model for everything
ZeroRouter	No clusters; mixes the two best-value models (paper's baseline)
K-NN	Uses the most similar validation prompts (paper's baseline)
UniRoute (K-means)	Our router
Production	Cost and similarity only (what actually happened)

Metrics: area under the cost–quality (deferral) curve; QNC, the lowest cost at which routing matches the best single model; cost saved at equal quality; quality per cluster and per category; routing share per model; semantic similarity to production's answers (the brief's metric); cost per 1,000 queries; p50/p90 latency; cache hit rate.

Simulating new models. We hide about a third of the models while choosing K and λ, then route only among them. This tests UniRoute's main claim.

Overfitting checks (all free). Locked test set; K by split-validation; $K = 1$ baseline; fingerprint stability; separability; validation-versus-test gap.

Success criteria.

- UniRoute's curve area $\geq$ ZeroRouter's and K-NN's on the test set.
- At least 40% cheaper than always-best at no more than 2 points lower quality.
- p90 latency no worse than always-best.
- Every number labeled measured or simulated, with confidence intervals.

Honest framing: with a fixed set of models the paper finds UniRoute only about as good as simpler routers (its App. G.5). Its advantage is adding models without retraining. We report the comparison either way.

8 Plan and budget

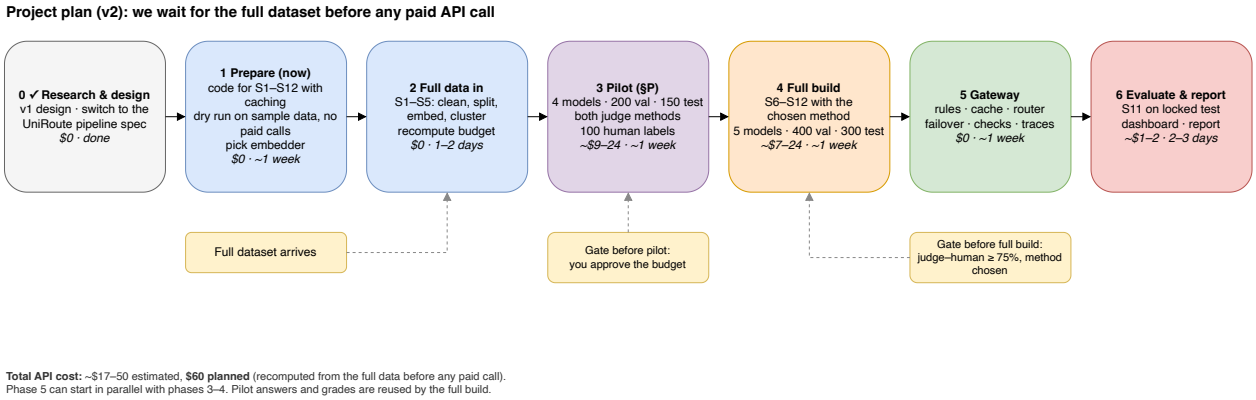


Figure 5: Project phases. No paid API call happens before the full dataset arrives.

Phase	What	API cost	Time
0 ✓	Research, v1 design, switch to the UniRoute pipeline specification	$0	done
1	Now, while waiting: code for S1–S12 with caching; dry run on the sample data with no paid calls; choose the embedder	$0	∼1 week
2	Full data arrives: S1–S5; recompute the budget	$0	1–2 days
3	Pilot: 4 models, 200 val, 150 test, both grading methods, 100 human labels	∼$9–24	∼1 week
4	Full build S6–S12 with the chosen method: 5 models, 400 val, 300 test	∼$7–24	∼1 week
5	Gateway: rules, cache, router, failover, checks, traces (can overlap 3–4)	$0	∼1 week
6	Evaluate on the locked test set; dashboard; report	∼$1–2	2–3 days
Total		**$17–50 (plan $60)**	**4–5 weeks after the data**

Gates. The team approves the recomputed budget before the pilot; the judge must agree with humans at least 75% of the time before the full build.

Budget item	Calls	Estimate
Pilot generation (3 cheap models + anchor, 350 prompts)	1,400	∼$3
Pilot judging (pointwise 1,400 + pairwise 2,100)	3,500	$6–21
Full-build generation (extra prompts, 5th model)	∼2,100	$3–4
Full-build judging, pointwise / pairwise	∼2,100 / 3,500	$3–13 / $6–21
Production quality on test (optional), live latency runs	300–600	$1–2
Total		**$17–50**

Per-call prices come from the sample's token counts (∼1,640 input and ∼174 output tokens per turn) and list prices; a judge call costs about $0.002–0.006. All estimates are recomputed from the full

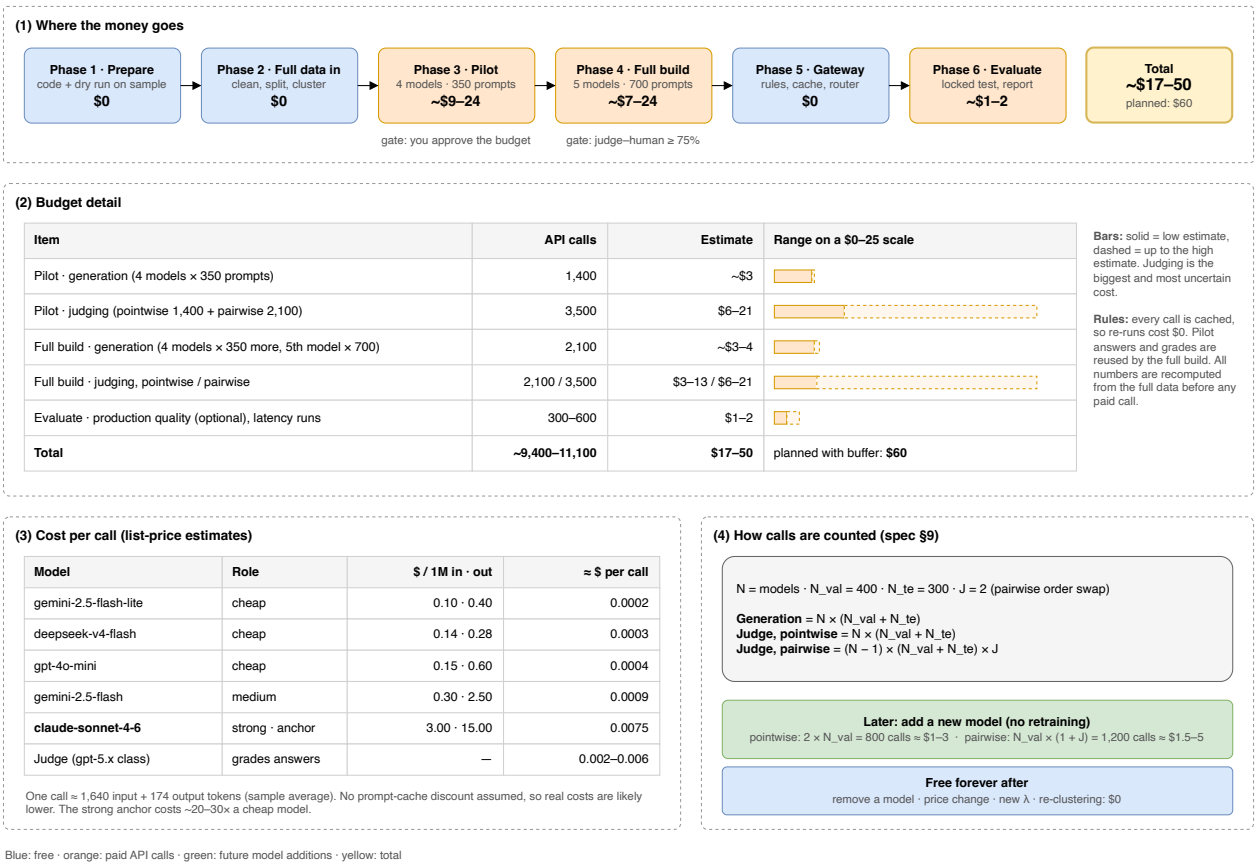


Figure 6: API costs: where the money goes, budget detail with low–high ranges, cost per call, and how calls are counted. Every call is cached, so re-runs cost $0.

data before any paid call.

Later upgrades

Upgrade	Why
LearnedMap (paper §5.2)	Soft cluster assignment; needs labels for 3–5 models on a few thousand train prompts
Safety-net judge on uncertain or sensitive messages	Extra protection where clusters are unsure
Cache-aware cost per message	Staying with one provider is cheaper thanks to prompt caching
Attribute-based clusters (paper App. G.4)	More robust when traffic changes
2–3 answers per (prompt, model); a second judge	Less noisy labels (DARS, EMNLP 2026)
Shared cache and breaker state (Redis)	Several gateway instances

9 Risks

Risk	Mitigation
Full dataset late or smaller than needed	Phase 1 is useful regardless; with less data, smaller validation/test sets and fewer clusters, stated in the report
Judge unreliable in Persian	Human agreement gate $\geq 75\%$; pilot criterion B
Labels cannot tell models apart	Pilot criterion D; raise τ or switch to pairwise
UniRoute no better than simpler routers	Reported honestly; keep the winner in the gateway
Only 5 models weakens the new-model test	Optionally add 1–3 candidates
Estimated prices	Recompute before any paid call
Credentials in the dataset	Redact before any API call; never commit data or generated answers

10 Decisions needed

1. **Full dataset:** when does it arrive, and how large is it?
2. **API access:** OpenRouter, direct provider keys, AvalAI, or Google Cloud?
3. **Budget:** approve about $60 in principle (final number after the full data arrives).
4. **Human calibration:** who labels 100 Persian answers in the pilot?
5. **Candidate pool:** keep 5 models, or add 1–3 for a stronger new-model test?
6. **Ask the course team** what the `default` model in the data means.
7. **Deadline and deliverable format.**

References

[1] Jitkrittum et al. Universal Model Routing for Efficient LLM Inference (UniRoute). ICLR 2026. https://arxiv.org/abs/2502.08773 — primary research

[2] Hu et al. RouterBench (ZeroRouter and K-NN baselines). 2024. https://arxiv.org/abs/2403.12031

[3] From Sampled Outcomes to Capability Distributions (DARS). EMNLP 2026. https://arxiv.org/abs/2606.06924

[4] Dynamic Model Routing and Cascading for Efficient LLM Inference: A Survey. 2026. https://arxiv.org/abs/2603.04445

[5] How Reliable is Multilingual LLM-as-a-Judge? EMNLP 2025 Findings. https://aclanthology.org/2025.findings-emnlp.587/

[6] RouterArena. ICLR 2026. https://arxiv.org/abs/2510.00202